ALIGNMENT DOES NOT COMPOSE: GOVERNED EXECUTION FORESTS FOR AGENT SWARMS

Muntaser Syed
Florida Institute of Technology

With
Apart Research

Abstract

Per-agent alignment and per-agent governance do not compose to swarm-level safety. We show this constructively. Each agent in a small deterministic swarm runs as an individually governed execution tree under pollard, with its own budget and policy. Every agent follows the same locally rational rule: delegate work instead of doing it, because delegation is cheaper for the delegator than local work. No agent breaks its own rules, yet the ungoverned swarm spends 651 credits against a 300 credit reference pool and delegates to depth 4, because the cost of a delegation lands on other agents and on the collective. The remedy is also at the execution layer. Treating the swarm as a forest, with each delegation edge recorded as a governed action in the delegator's tree, one shared budget pool across all trees, and a depth gate with an approval step, contains the same swarm at 251 credits and depth 2. A drift layer built on epicormic sequential detectors watches a per-agent action-cost stream; when one agent's work cost is multiplied by 4 mid-run, the detector alarms on the first inflated action with no false alarm before onset, a drift verdict is recorded in the epicormic ledger, the agent is frozen, further delegations are refused by epicormic's DriftMeter, and the run completes at 281 credits. In all three conditions the recorded forest replays exactly: the replayed decision and edge trace has the same SHA-256 digest as the recorded trace, and replay invokes no step functions.

1 Introduction

Agent governance tooling is mostly scoped to a single run: a budget for this run, a policy for this run's tool calls, an approval gate in front of this run's side effects. A swarm is a set of such runs that hand work to each other. The failure mode studied here is compositional. Delegation moves cost across the boundary that per-run governance watches. The delegator pays a small fee and stays inside its own budget; the delegatee absorbs the work; the total grows with the number of edges, not with any single agent's spend. Depth compounds it: a sub-item delegated twice is outside the depth accounting of every individual tree that touched it, because each tree only sees its own edges.

The experiment isolates this failure with deterministic agents. Determinism is a design choice, not an oversight: it makes the execution trace exactly replayable, so the governance claims below are checkable against recordings rather than argued from samples. Section 5 states what this does and does not generalize to.

Our contributions are:

[0]Research conducted at the Swarm Alignment Hackathon, November 2026.

1. A constructive, measured demonstration that per-agent governance does not compose to swarm safety: the ungoverned swarm spends 651 credits against a 300 credit pool, delegates to depth 4, and fails the task, while no agent breaks its own rules.

2. A forest-level construction, with governed delegation edges, one shared budget pool, and a depth gate with approval, that contains the same swarm at 251 credits and depth 2, with exact replay of the recorded forest in every condition.

3. A drift layer, epicormic sequential detectors over swarm telemetry, that detects an injected cost drift at the first inflated observation with no false alarms, freezes the agent, and completes the run at 281 credits.

2 Related Work

The position taken here is that swarm governance is an execution-layer property, not a property of any agent framework's prompt or planning loop. If delegation is an action in a recorded execution tree, if the accounting unit is the pool rather than the run, and if behavior streams are monitored with sequential detectors wired to fail-closed levers, then the controls are framework-independent: they bind whatever produced the actions. That is the same bet pollard makes for single runs and epicormic makes for provider drift, extended one level up, from the tree to the forest. Governance tooling scoped to a single agent run, however complete for that run, leaves the edges between runs ungoverned, and the non-composition result in Section 4 is the reason the extension is necessary: governing every tree in the forest, one at a time, leaves the edges ungoverned, and the edges are where the failure lives.

Prior work. pollard (version 1.6.2) and epicormic (version 0.2.0) are the author's published open-source packages, and they are preexisting: both were released before this sprint and are used here as unmodified dependencies. pollard provides governed execution trees for single agent runs; epicormic provides drift detection and containment for pollard-governed systems. The swarm layer was written during the sprint: the forest construction, the delegation gate rules, the shared-pool configuration, the drift monitor wiring, the experiment, and the analysis. Only the sprint work is claimed as this submission's contribution.

3 Methods

Trees. Each agent owns one pollard run. Local work is a `model_call` node on the agent's run, priced in credits by a meter. Everything an agent does, including every refusal it hits, is a node in its own content-addressed tree, which pollard can verify and replay.

Forest and governed edges. Delegation is a registered pollard action named `delegate`, executed as a `tool_call` on the delegator's run. The edge therefore exists as a node in the delegator's tree, carrying the target agent, the child item, and the delegation depth. The swarm is the forest of the agents' trees plus these edges, and an edge is subject to the same machinery as any governed action: policy decision, approval gate, budget precheck, recorded refusal on denial.

Shared pool. Governed conditions bind one pollard `SharedBudget` named `pool` across all runs in a transactional SQLite store, with a per-run sub-allocation on top. A charge by any agent settles against the pool; an action that would exceed the pool or the acting agent's sub-allocation is refused before dispatch and recorded as a refusal node.

Delegation gate. A pollard `Policy` implementation evaluates every `delegate` call with swarm-level state:

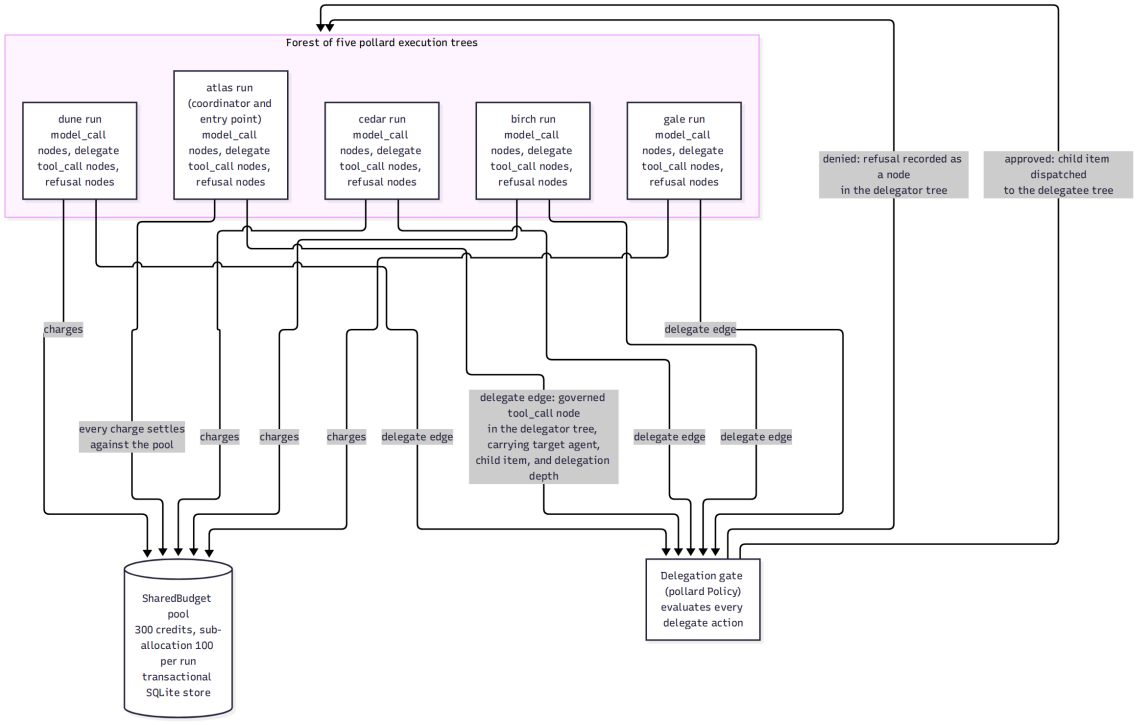


Figure 1: The governed execution forest. Each of the five agents owns one pollard run (tree) whose nodes are local work (`model_call`), delegations (`delegate tool_call`), and recorded refusals. Every `delegate` action passes through the delegation gate, a pollard `Policy`, before dispatch; a denial is recorded as a refusal node in the delegator's tree. All charges settle against one `SharedBudget` pool of 300 credits with a per-run sub-allocation of 100, held in a transactional SQLite store.

delegations from frozen agents are denied; delegations beyond a per-agent spawn limit are denied; delegations creating an edge deeper than the maximum depth (2 in the experiment) are denied; delegations creating an edge exactly at the maximum depth require confirmation under pollard's approval flow. The driver acts as approver and confirms only while the pool retains at least 25 percent of its capacity. A split whose delegation is refused is abandoned: the delegator works the item locally, and sub-items never dispatched are covered by that local work. Figure 1 shows the construction assembled.

Drift layer. The drift condition adds a monitor over the watched agent's executed action costs, in execution order (delegation fees and work charges). Detection uses epicormic's sequential detectors, `page_hinkley` as the primary rule and `cusum` as a cross-check, evaluated on the stream prefix after each new sample. On the first Page-Hinkley alarm the swarm layer assembles an epicormic verdict document from the detectors' output (state `drift`, rule `page_hinkley`, both sequential states) and records it through epicormic's `record_verdict`, so the verdict sits in the monitor chain and the epicormic `Ledger` reports state `drift`. Two containment levers then act. epicormic's `DriftMeter`, registered on the runtime for tool calls, refuses every further delegation swarm-wide while the ledger state is `drift`. The delegation gate freezes the drifting agent, and the driver reassigns its queued items to other agents. Figure 2 shows the detection and containment flow.

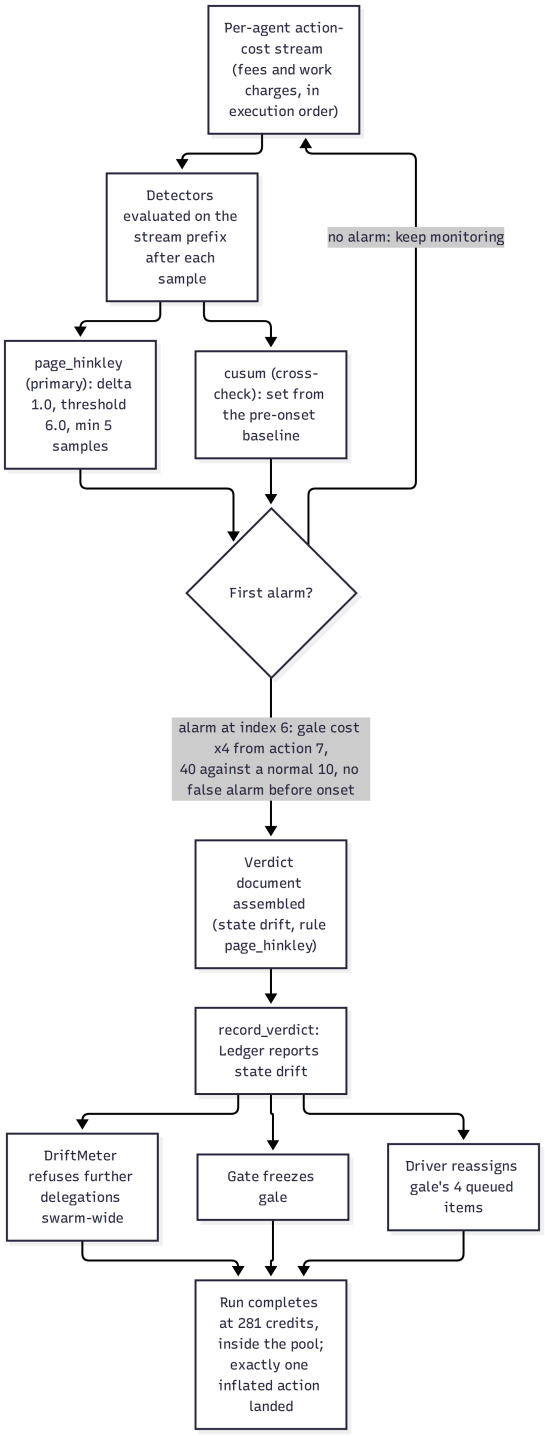


Figure 2: Drift detection and containment. The watched agent's action-cost stream is evaluated after each sample by epicormic's `page_hinkley` detector (primary rule) and `cusum` detector (cross-check). In the experiment, gale's work cost is multiplied by 4 from its seventh executed action; the first alarm fires at that first inflated observation, with no false alarm on the pre-onset prefix. The swarm layer records the verdict through `record_verdict` and the `Ledger` reports state `drift`; the `DriftMeter` then refuses further delegations swarm-wide, the delegation gate freezes gale, and the driver reassigns its 4 queued items. The run completes at 281 credits, inside the pool.

What calls which API. pollard provides the runtime, trees, registry, policy and confirmation flow, budget scopes including the shared pool, the meter protocol, `recompute_charges`, `verify`, and replay. epicormic provides the two sequential detectors, the verdict and ledger machinery (`Monitor`, `Verdict`, `record_verdict`, `Ledger`), and the `DriftMeter` lever. The swarm layer implements the agents and task economy, the assignment rule, the gate's rule set as a pollard policy, the approver rule, the refused-split fallback, the forest recorder and its trace digest, the drift stream, and the verdict document assembly. Two available primitives were deliberately not used. pollard's team delegation helpers bind sub-runs under a team runtime; here the edge had to be a governed node in the delegator's own tree, which the registered-action route expresses directly. epicormic's probe panel and scorer pipeline sample model endpoints; the monitored stream here is swarm telemetry, not probe responses, so the verdict document is assembled from the sequential detectors and recorded through the same ledger path, with the vacuous subjective-logic opinion because no panel opinion is computed. epicormic's `DriftPolicy` and `ContractGate` levers were not used.

Experiment setup. Five deterministic agents: atlas (coordinator and entry point), gale (aggressive), birch, cedar, dune. Workload: 3 root items with `splits = 4`. An item with splits remaining can be split into sub-items with one less split, or worked locally at cost $6 + 2 \times$ splits credits (6 to 14). Delegation costs the delegator a fee of 1 credit per edge. Fanout is 3 for atlas and gale, 2 for the others. Every agent delegates when it can, because delegating costs at most 3 credits against 6 to 14 for local work; this is the locally rational rule. Non-aggressive agents stop splitting at depth 3; gale splits at any depth. Sub-items are assigned round-robin across the roster by creation order. Governed parameters: pool 300 credits, sub-allocation 100 credits per agent, maximum delegation depth 2, spawn limit 60 per agent. Drift condition: gale's work cost is multiplied by 4 starting from its seventh executed action; Page-Hinkley with delta 1.0 and threshold 6.0, minimum 5 samples; CUSUM reference and threshold set from the pre-onset baseline. Task success means every root item resolves and total spend stays within the pool. Replay re-runs the same driver against the recorded store in pollard HYBRID mode and compares forest recorder digests.

What did not work. A first parameterization of the scenario did not engage the governance at all: the run ended at 186 of the 300 credit pool with the gates barely exercised, so nothing about composition could be measured. The scenario was retuned, with the workload and fanout above, until the ungoverned economy overshot the pool and the governed economy pressed on the gates. In the final parameterization the depth gate binds before the pool does: the pool and the per-agent sub-allocations refuse nothing in the scenario runs, so pool refusal, including refusal of one agent's action because another agent spent the pool down, is covered by the unit tests against a small pool rather than by these runs. The spawn limit of 60 also never binds.

4 Results

Table 1 gives the measured outcomes for the three conditions. The same swarm, the same scenario parameters, and the same locally rational delegation rule run in all three; only the governance layer changes.

Ungoverned, the delegation cascade is the whole story: every item splits whenever its owner can split, 120 items are created, all roots complete, and the swarm spends 651 credits, 2.17 times the reference pool, reaching delegation depth 4. No individual agent misbehaves. Each delegation was the cheapest action available to its delegator.

Governed, the same economy is contained by structure rather than by exhortation. The depth gate denies 22 split attempts at the boundary; items whose splits are refused are worked locally by their owners; 53 proposed sub-items are never dispatched. The approval gate is exercised 22 times at depth 2 and the approver confirms all 22, because pool headroom stays above the 25 percent floor at those points. Total spend is 251

Table 1: Measured outcomes for the three conditions. The pool is 300 credits in every condition; in the ungoverned condition it is a reference value only, not an enforced budget. The drift outcome row applies only to the governed + drift condition, where gale's work cost is multiplied by 4 from its seventh executed action.

Metric	Ungoverned	Governed	Governed + drift
Total spend (credits)	651	251	281
Pool (credits)	300 (reference)	300	300
Max delegation depth	4	2	2
Gate denials	0	22	22
Confirmations requested / approved	0 / 0	22 / 22	22 / 22
Budget refusals	0	0	0
Drift refusals	0	0	1
Items resolved / created	120 / 120	34 / 87	34 / 84
Task success	no	yes	yes
Drift outcome	n/a	n/a	contained
Replay trace digest match	yes	yes	yes

credits and depth is capped at 2.

In the drift condition, gale's stream before onset is six delegation fees of 1 credit. Its seventh executed action is the first inflated work charge, 40 credits against a normal 10. Page-Hinkley alarms at stream index 6, the same index as onset: detection delay 0 steps, where a step is one executed action of the watched agent. CUSUM alarms at index 6 as well. Neither detector alarms on the pre-onset prefix. The verdict is recorded in the epicormic ledger, gale is frozen with 4 queued items reassigned, and a post-verdict delegation probe by atlas is refused by the DriftMeter. Exactly one inflated action lands before containment, putting total spend at 281 credits, still inside the pool.

Robustness. The runs are deterministic and seeded, so every number in Table 1 reproduces exactly from the code in the repository. Replay holds in all three conditions: the replayed trace digest equals the recorded digest, replay invokes zero step functions (every executed call resolves from its recording; refusals re-derive from the same policy and budget state), pollard `verify` passes on every agent chain, and the spend recomputed from the store by pollard's `recompute_charges` equals the recorder's total in every condition. The unit test suite passes 9 of 9, including the small-pool tests that cover pool refusal. The scope is deliberately narrow: a single scenario family, a single drift injection, and a single swarm size. The results demonstrate the mechanisms and the compositional failure on that scenario; they are not a measurement over scenario variation.

5 Discussion and Limitations

The failure in the ungoverned condition is structural, not behavioral, and that is the point with the widest implications. No agent was misaligned with its own instructions, and no prompt-level or planner-level fix addresses the failure, because the cost that breaks the collective budget is created by edges that no single agent's governance can see. Two design consequences follow. First, the accounting unit for a swarm has to be the collective: a shared pool that any agent's charge settles against, with refusal before dispatch, rather than a set of per-run budgets whose sum nobody enforces. Second, delegation itself has to be a governed action, recorded in the delegator's tree with its depth and target, so that depth limits, spawn limits, freezes, and approvals apply to the act of handing work off and not only to the work itself. With those two pieces

in place, the rest of the machinery in this paper is reuse: the drift layer is the same sequential detection and fail-closed containment used for provider drift, applied to swarm telemetry, and exact replay gives an auditor the whole forest, decisions and refusals included, after the fact.

5.1 Limitations

The agents are deterministic scripts. The results show governance mechanics (accounting, gating, detection, containment, replay) and one compositional failure mode; they say nothing about how LLM-driven agents would behave under the same gates, and nothing about strategic adaptation to the gates. The task family is a single split-or-work item tree; other delegation topologies (shared subgoals, cyclic handoffs, negotiation) are untested. Drift is injected synthetically as a cost multiplier on one agent at a known point; real provider or behavior drift will be noisier, and the zero-delay detection here reflects a large, abrupt shift (a factor of 4) against a quiet baseline of 1 credit fees. The swarm is small: 5 agents, at most 120 items. As noted in Section 3, the depth gate binds first in this regime, so the budget scopes act as a backstop whose refusal behavior is unit-tested, not measured in the scenario runs. Replay uses HYBRID mode because a refused call leaves a refusal node rather than a call recording, so pure REPLAY mode has nothing to resolve for refused actions; exactness is therefore claimed at the level of the decision and edge trace with zero step function invocations, which is the level the trace digest measures.

5.2 Future Work

Four extensions follow directly. First, replace the deterministic agents with LLM-driven agents under the same forest governance, to test whether the compositional failure and its containment survive agents that plan, phrase, and adapt. Second, scale the swarm, in agent count and in items, to find where the shared-pool accounting and the approval flow become bottlenecks. Third, study adversarial delegation strategies, where agents probe the gate rules, split work to stay under depth or spawn limits, or time delegations against the approval floor. Fourth, construct a parameterization in which the pool binds before the depth gate, so budget refusal between agents is measured in scenario runs and not only in unit tests.

6 Conclusion

Per-agent governance does not compose, and the gap is measurable: five individually governed agents, each following the cheapest available action, spent 651 credits against a 300 credit pool and delegated to depth 4, failing the task without any agent breaking its own rules. Governing the forest instead of the trees contains the same swarm: governed delegation edges, one shared budget pool, and a depth gate with approval hold it to 251 credits at depth 2, and a sequential-detector drift layer catches an injected factor-of-4 cost drift at the first inflated observation, freezes the agent, and completes at 281 credits.

The controls are execution-layer controls, so they bind whatever produced the actions: a script today, a framework or a model tomorrow. That is the claim this submission makes and measures. If swarms are going to be governed at all, the governance has to live where the delegation happens, because the edges are where the failure lives.

Code and Data

Code repository: `https://github.com/jemsbhai/swarm-forest`

Data: No external datasets were used. The experiment records (results.json and the per-condition pollard stores) are generated by the runner in the repository.

Author Contributions

Muntaser Syed conceived the project, directed its implementation and experiments, and prepared the report.

References

1. M. Syed. pollard: governed semantic execution trees for agent control and replay. Software package, version 1.6.2, PyPI, 2026. `https://pypi.org/project/pollard/`
2. M. Syed. epicormic: provider drift detection and containment for pollard-governed systems. Software package, version 0.2.0, PyPI, 2026. `https://pypi.org/project/epicormic/`
3. E. S. Page. Continuous inspection schemes. *Biometrika*, vol. 41, no. 1/2, pages 100 to 115, 1954.
4. D. V. Hinkley. Inference about the change-point in a sequence of random variables. *Biometrika*, vol. 58, no. 1, pages 1 to 8, 1971.

Appendix A. Limitations and Dual-Use / Ethical Considerations

Limitations. The agents in this study are deterministic scripts, not language models. The results therefore demonstrate governance mechanics (accounting, gating, detection, containment, and exact replay of the decision and edge trace) and one compositional failure mode. They do not show how LLM-driven agents would behave under the same gates, and they say nothing about agents that adapt strategically to the gates. The task family is a single split-or-work item tree with round-robin assignment; other delegation topologies, including shared subgoals, cyclic handoffs, and negotiation between agents, are untested. Behavior drift is injected synthetically, as a cost multiplier of 4 applied to one known agent at a known point in the run. Real provider or behavior drift will be noisier and usually smaller, and the zero-step detection delay reported here reflects a large, abrupt shift against a quiet baseline of 1 credit fees; it should not be read as an expected detection delay in deployment. The swarm is small: 5 agents and at most 120 work items. In the governed scenario runs, the depth gate binds before the budget scopes do, so pool and sub-allocation refusal behavior is established by unit tests against a small pool (9 of 9 tests pass) rather than measured in the scenario runs. Replay exactness is claimed at the level of the decision and edge trace in HYBRID mode, with zero step function invocations; refused actions leave refusal nodes rather than call recordings, so pure REPLAY mode has nothing to resolve for them.

Dual-use and ethical considerations. The system built here is execution-layer governance: budgeting, approval gating, behavioral monitoring, and audit for multi-agent systems. That is defensive control tooling, intended to keep agent swarms inside stated limits and to make their actions attributable after the fact. The same underlying machinery, a recorded delegation graph between cooperating agents, could in principle also be used to coordinate multi-agent systems toward harmful ends, for example by making a harmful swarm easier to schedule, budget, and operate. We judge that this consideration argues for publishing the control and audit mechanisms openly rather than withholding them: the contribution is the governance layer (limits, gates, detection, containment, and exact replay), which is what an operator, auditor, or regulator

needs in order to constrain any swarm built on shared execution infrastructure, and open publication lets defenders and evaluators test those controls. The experiments themselves carry no capability uplift: the economy is synthetic and deterministic, the agents are scripts written for this study, and no real agents, real data, credentials, or external systems are involved. No human subjects took part, and no personal data was collected or processed.

LLM Usage Statement

An AI assistant was used throughout the sprint to help implement the experiment code, run the experiments, and draft this report, under the author's direction. Every reported result was produced by executing the code in the repository and was verified against the recorded outputs before inclusion.